

图 7.10 展示了三个任务 A、B、C 基于 MLFQ 策略的执行流。其中任务 A 是一个长任务，而任务 B 和 C 的运行时间相对较短。在初始阶段，任务 A 和 B 进入系统，并且被分配最高的优先级。当它们在队列 0（最高优先级队列）中执行至最大运行时间后，调度器会降低它们的优先级，移至队列 1。任务 B 在队列 1 执行结束，而任务 A 一直在执行，因此任务 A 的优先级会被进一步降低并移至队列 2。当任务 A 在队列 2 执行到第二个时间片时，任务 C 进入系统，它被调度器分配最高的优先级，抢占任务 A 的执行。过了一定的时间后，到达调度器定时提升任务优先级的时间点，任务 A 和 C 都被移至队列 0 执行。任务 C 随后在队列 0 的第一个时间片结束自己的执行，长任务 A 继续执行，并最终再次回到最低优先级的队列 2。

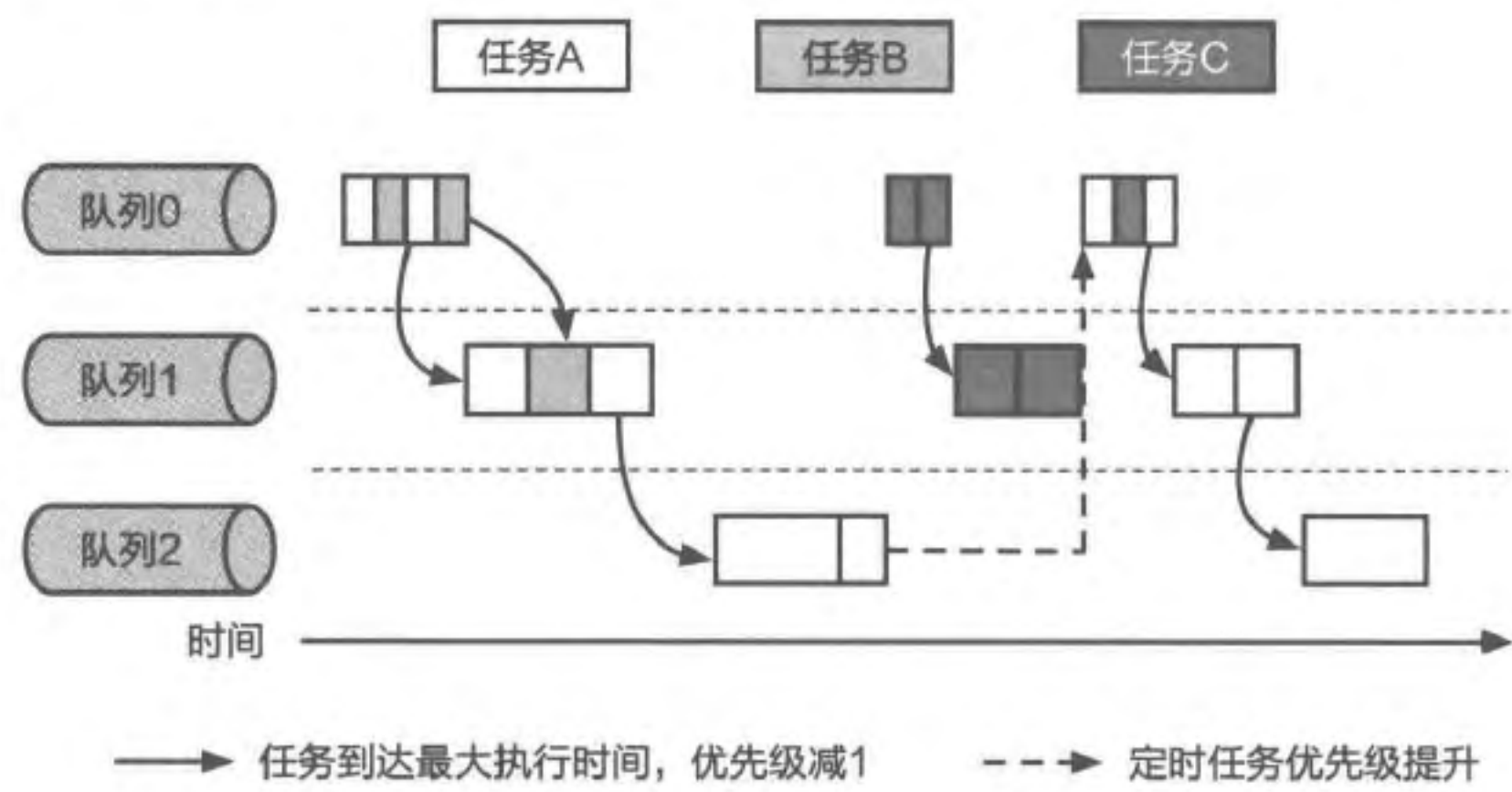


图 7.10 MLFQ 策略的执行流

在 MLFQ 策略的设计中，任务的优先级会被动态地提升（Boost）和降低（Penalty，对应“惩罚”）。提升和降低是用于动态调节任务优先级的机制，被应用在包括 Linux 在内的许多操作系统中。系统可以启发式地确定哪些任务需要被提升或降低优先级，以及优先级的变化量应该是多少，进而调整优先级调度的结果。然而，这些启发式方法在提升调度效果的同时，也可能造成实现复杂的问题。

在 MLFQ 策略的具体实现中，有许多调度参数需要调整，例如优先级队列的数量、每个优先级队列采用的时间片、任务在每个优先级队列的最大运行时间以及调度器定时提升优先级的时间间隔。如果参数使用不当，就可能达不到预期的调度效果。比如，如果提升优先级的时间间隔过短，那么所有任务都会保持在最高优先级的队列中，退化成 RR 策略；而如果该间隔过长，很可能会导致长任务保持在最低优先级队列中，很长时间内无法执行。因此，虽然 MLFQ 调度策略的设计非常巧妙，但开发者仍然需要花费很大的努力，对系统整体有细致的了解，才可以实现出有效的基于 MLFQ 策略的调度器。

MLFQ 策略通过记录任务运行时间的方式，动态地调配任务的优先级。它能达到与 SJF 策略和 STCF 策略类似的低平均周转时间，同时还能保证任务的响应时间，并且避

免了任务饥饿。以 MLFQ 为基础的调度策略在很多操作系统中都得到了应用，例如早期 Linux、Windows 和 macOS^①。

7.4.3 优先级调度策略的比较

本章介绍的 HRRN 策略说明，设置优先级调度策略的优先级是需要考虑多种因素并经过深思熟虑的。在此基础上，本章分别介绍了静态优先级策略 MLQ 和动态优先级策略 MLFQ，展示了实际调度策略如何设置任务的优先级。根据优先级设置策略的不同，它们的区别也显而易见：HRRN 策略主要希望融合 FCFS 与 SJF 策略的优点，在短任务优先的同时保证长任务不会饿死，所以其优先级设置会同时考虑任务的运行时间和等待时间；MLQ 策略主要关注任务已知且种类划分明确的场景，所以其优先级会根据任务种类而设置；而 MLFQ 策略则是一种通用的优先级调度，其不同的规则实际上是为了满足不同的设计考量。因此，在选择优先级调度策略时，读者应该首先明确适合目标场景的特征与调度指标，这样才能设计出合适的优先级调度策略。

7.5 公平共享调度策略

本节主要知识点

- 为什么需要公平共享调度？什么是份额？
- 有哪些典型的公平共享调度策略？
- 份额与优先级的异同是什么？

优先级调度被广泛应用在大量场景中，然而优先级调度也并非在所有场景中都能适用。以资源共享场景为例，当用户共享服务器时，一个默认假设是用户使用的资源数（例如 CPU 时间）与所花费的钱是成正比的。现在，假设小明和小红合租了一台云计算平台上的单核服务器，他们商定平分 CPU 时间，并且小明同时开启三个任务（任务 A、B、C），而小红只开启一个任务（任务 D）。那么有什么调度策略能够保证小明的三个任务和小红的一个任务平分 CPU 时间呢？

假设这些任务的优先级相同，并且采用 RR 策略进行“公平”的调度。那么由于此时小明开启三个任务，会占据 75% 的 CPU 时间，而小红则只能使用剩下的 25%，因此 RR 策略不能满足两人对于 CPU 资源平分的约定。

小明和小红决定尝试优先级调度，小明的三个任务优先级被设置为 1（低优先级），而小红的一个任务优先级被设置为 3（高优先级），这样的优先级分配从数值上体现了用

^① 资料来源于维基百科，参见 [https://en.wikipedia.org/wiki/Scheduling_\(computing\)](https://en.wikipedia.org/wiki/Scheduling_(computing))。

用户对资源占比的预期 [小明与小红的 CPU 时间占比为 $(1+1+1):3$]。然而实际调度中，在没有防止低优先级任务饥饿机制的情况下，小红的任务会持续执行直到结束。我们发现，优先级的数值仅仅是用于比较优先级高低而存在的，无法反映单位时间内一个任务可以占用的 CPU 时间比例。因此优先级调度也无法保证任务对资源使用的实际比例。

在考虑到资源的使用情况时，用户看重的已经不再是平均周转时间或者响应时间，而是自己在总资源中的占用比例，用户希望：

- “优先级”为 3 的任务占用的 CPU 时间是“优先级”为 1 的任务的 3 倍，是“优先级”为 6 的任务的一半。
- 系统资源的分配支持以用户或一组任务为粒度。

为了满足以上的需求，公平共享调度[⊖]（Fair-Share Scheduling）应运而生，这类调度器会量化任务对系统资源的占用比例，从而实现对于资源的公平调度。我们将以份额（Share）来量化每个任务对 CPU 时间的使用。图 7.11 给出了小明和小红使用份额的例子，假设当前 CPU 核心只负责处理他们两人的请求。从小明和小红的视角来看，假设每人都拥有 40 的份额，即每人应使用一半的 CPU 时间。任务份额并非系统直接给定的，而是两人从自己的份额中选取一部分交由各自的任務。图 7.12 给出了根据以上例子，理想情况下的公平共享调度应达到的 CPU 时间占比。例如，小明的份额占了系统总份额的 50%，而任务 B 的份额占了小明总份额的 50%，因此任务 B 的实际 CPU 占比是 $50\% \times 50\% = 25\%$ 。

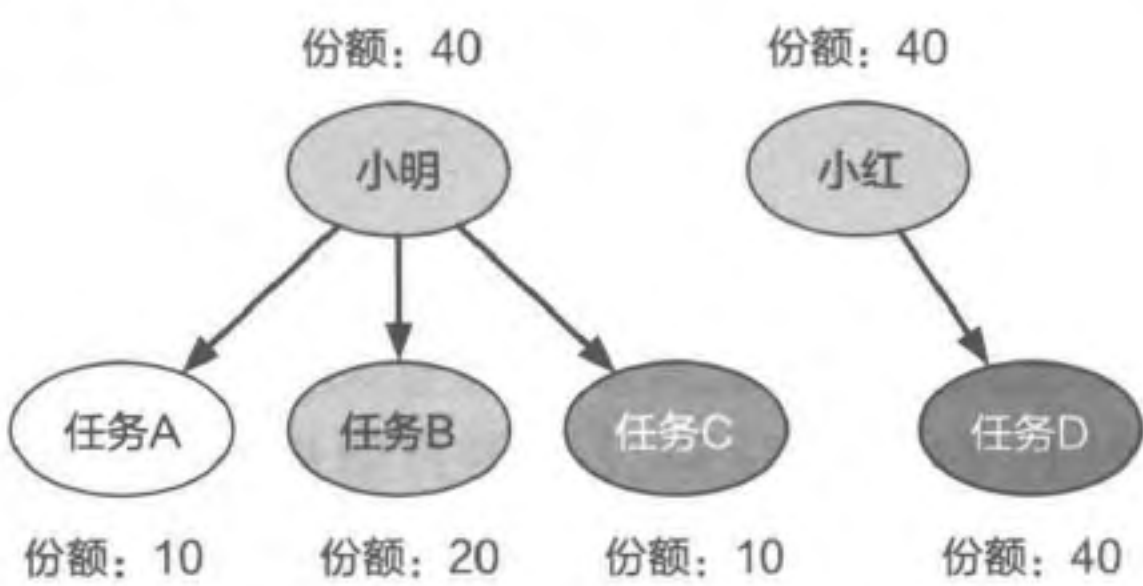


图 7.11 小明和小红使用份额的例子

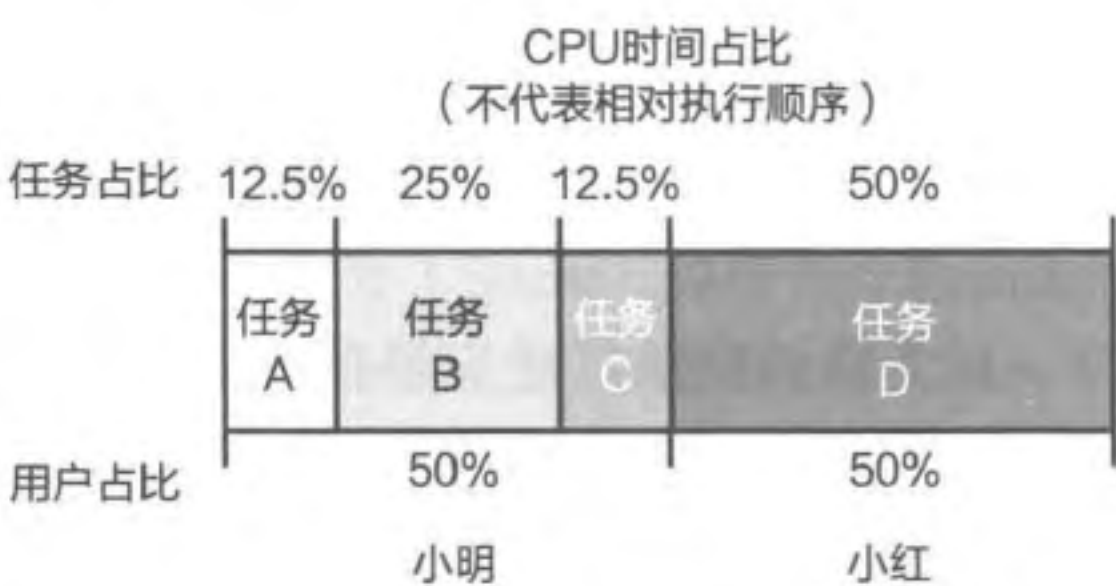


图 7.12 小明和小红发起的任务在公平共享调度下的理论 CPU 时间占比

以上例子反映了用户不同于此前的需求，他们可能需要一个直接或间接的参数来控制任务的资源使用情况。而份额支持层级化的分配方式，可以直观地体现每个任务理论上对资源的使用情况。在实际场景中，为了方便管理，可以将任务分组（称为一个任务组）。以组为单位分配份额，任务会在组内部进一步分摊该组所拥有的份额。

在了解了份额这一概念后，下面将介绍两个经典的公平共享调度策略，彩票调度和步幅调度。

⊖ 公平共享调度的“公平”指的是任务对资源的使用符合用户预期。

7.5.1 彩票调度

公平共享调度策略的设计思路是根据每个任务所占用的份额，成比例地分配 CPU 时间。这类调度策略需要解决的问题是如何高效地达到公平共享的目标。本节将介绍著名的彩票调度 (Lottery Scheduling) 策略^[3]。其中，彩票对应份额。假设现在班级举行抽奖活动，小明和小红分别拥有 3 张和 6 张彩票，其他同学一共拥有 21 张彩票。活动规则是在这 30 张彩票中随机抽取一张，抽中彩票的归属者即中奖。可以算出小明和小红单次抽奖的中奖概率分别为 10% 和 20%。如果将抽奖的次数上调，那么他们的中奖频率也将趋近于 10% 和 20%。

彩票调度策略的思想与实际的彩票抽奖类似。在该策略中，份额相当于每个同学拥有的彩票。在每次调度时，任务会根据随机数 (彩票开奖) 确定是否被调度。任务所占的份额越大，随机数就越有可能落在它的份额之内，因此就越有可能被调度。随着调度 (开奖) 的次数不断增加，任务被调度的次数在总调度次数中的比例也将趋近于该任务所拥有的份额占总份额的比例。

代码片段 7.2 展示了基于彩票调度的伪代码，对应 7.1.2 节描述的编程接口 (代码片段 7.1) 中，从运行队列选择下一个被调度任务的逻辑。调度器保留当前的总彩票数 (total_tickets, 第 5 行)，所有任务存储在一个运行队列 (run_queue, 第 9 行) 中，每个任务记录自己占用的彩票数 (ticket, 第 10 行)。在调度时，调度器在总彩票数的范围内随机生成一个中奖彩票号码 (R, 第 5 行)。接着，调度器遍历任务列表中的元素，并使用一个计数器 (sum, 第 3 行) 记录已经遍历的任务的彩票数总和 (包括当前正在遍历的任务的份额, 第 10 行)，如果这个总和超过了随机数 (R, 第 11 行)，则返回当前被选择的任务 (第 13 行)。

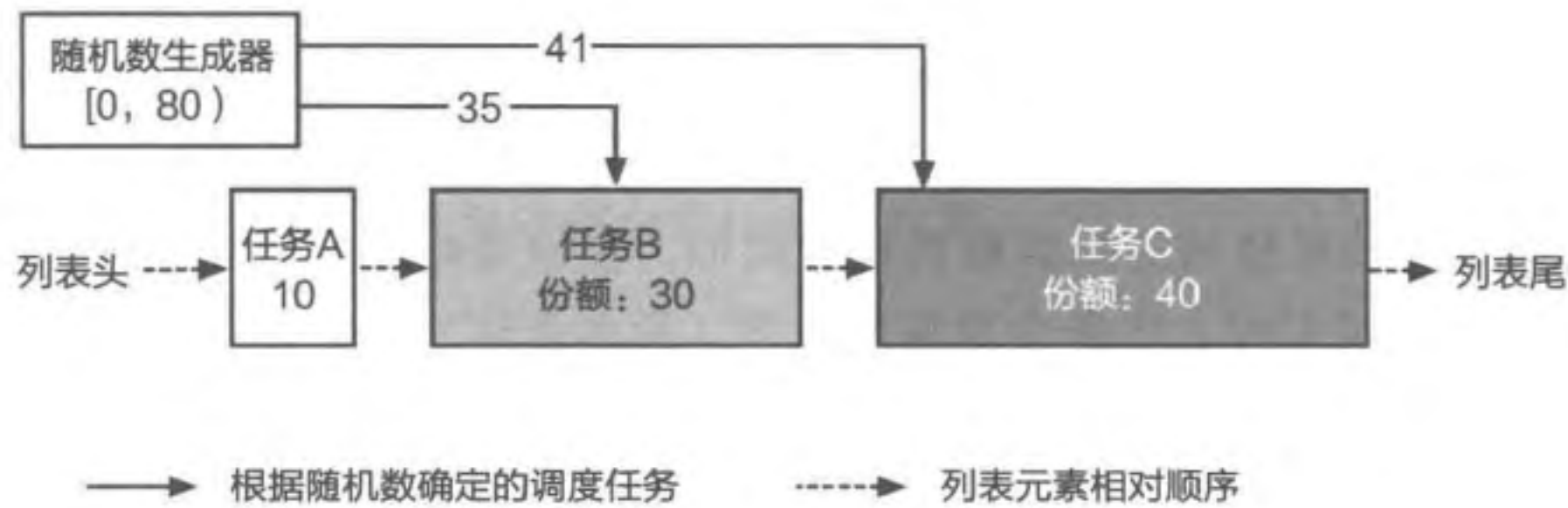
代码片段 7.2 彩票调度的伪代码

```

1 struct process* sched_dequeue(void)
2 {
3     int sum = 0;
4     // 生成随机数
5     int R = random(0, total_tickets);
6
7     struct process* proc;
8     // 遍历任务列表
9     for (proc in run_queue) {
10         sum += proc->ticket;
11         if (R < sum) {
12             // 返回被选择的任务
13             return proc;
14         }
15     }
16     // 此处代码不应被执行
17     return NULL;
18 }

```

以图 7.13 中的随机数 41 为例。调度器从列表头开始，首先遍历到任务 A。因为任务 A 的彩票数为 10，所以计数器从 0 加到 10，但是仍然小于 41，因此调度器继续遍历列表。当遍历到任务 B 时，计数器的值为 40，仍然小于 41。最后，当调度器遍历到任务 C 时，计数器的值为 80，大于 41，因此调度器选用任务 C 进行调度。类似地，当随机数为 35 时，彩票调度策略将选用任务 B 进行调度。



彩票调度策略一般不要求列表中的任务按照持有的彩票数量排序，因为影响任务被调度概率的因素是每个任务持有的份额，而非任务在列表中的顺序。不过，如果将任务按照持有的份额数量从高到低进行排序，则可以降低决策时列表的查找次数。仍然以图 7.13 中的随机数 35 为例，如果列表按份额数量从低到高排序，则需要两次列表查找才能确定调度的是任务 B。现在假设列表按份额数量从高到低排序，则任务应以 C、B、A 的顺序存储在列表中，随机数 35 在第一次查找时，就可以决定调度任务 C。

除了彩票调度的基本算法外，还有一些优化方法可用于解决不同的问题：

- 彩票转让 (Ticket Transfer)。在系统的实际运行中，一个份额大的任务 A 可能在等待份额小的任务 B 所持有的锁。彩票调度会让 A 有更高的概率运行，然而任务 A 申请不到任务 B 的锁，会造成 CPU 资源的浪费。针对上述问题，任务 A 可以将自己的份额转让给任务 B。在转让后，任务 B 将拥有更高的 CPU 占比份额，有更高的概率被运行，减少了资源浪费。
- 彩票货币 (Ticket Currency)。用户或者任务组在为子任务分配彩票时，可以采用自己的计算方式，类似于发行了自己的货币。假设现在小明和小红分别有 60 张和 40 张彩票，且他们各自的任務分别组成了一个任务组。小明可以设置自己总共有 400 货币，并进一步把 100 货币分配给任务 A，另外 300 货币分配给任务 B。那么任务 A 的实际彩票为 $60 \times (100/400) = 15$ 张，而任务 B 则为 45 张。那么为什么不直接将 15 张和 45 张彩票分配给两个任务，还要“多此一举”呢？试想如果此时小明的 20 张彩票被转让给了小红，在没有彩票货币机制的情况下，为了保证任务 A 和任务 B 的彩票之和为 40，就需要专门修改它们记录的彩票数量。而在使用彩票货币的情况下，由于任务 A 和任务 B 对应的彩票货币数不变，因此无须

进行修改，只需要对小明的任务组持有的彩票数进行修改即可。总之，通过添加一层彩票货币的抽象，可以让任务组更加灵活地修改自己持有的份额，避免影响从属于它们的任务。

- 彩票通胀 (Ticket Inflation)。彩票通胀的思想是给任务一定自由度，允许任务根据当前对 CPU 资源的需求决定自己的份额。这种动态调整份额的方式可以应对变化的应用场景，让需要资源的任务动态地申请更多的资源，而不需要资源的任务则可以释放自己的资源。但是，彩票通胀需要多个任务间互相信任，如果有任务恶意地提升自己的彩票量，那么最坏情况下它会占用绝大部分的 CPU 时间。

随机数带来的问题。彩票调度通过使用随机数的方式实现了一个简单且近似于公平共享的调度器。然而，随机数导致某一任务占用 CPU 时间的比例，需要在该任务经历多次调度后，才能趋近于该任务的份额在所有任务总份额中的比例。即只有调度次数足够多，彩票调度的效果才接近公平。假设同一时间内，两位用户分别发起了一个任务，并期望平分 CPU 时间，那么理论上两个任务被调度次数的比例应该近似为 1:1 (假设两个任务的时间片相同)。图 7.14 展示了模拟的彩票调度中，随着总调度次数的增加，两个任务被调度次数的比例 (在这里简称为“比例”)。在模拟中，总调度次数小于 16 时，比例一直与理想的 1:1 相差较多。总调度次数大于等于 16 时，比例近似于 1:1，但仍然有一定波动。对于运行时间比较短的任务，如果调度次数过少，彩票调度实际分配给任务的 CPU 份额可能因为随机性而与预期不同，不一定能保证公平。

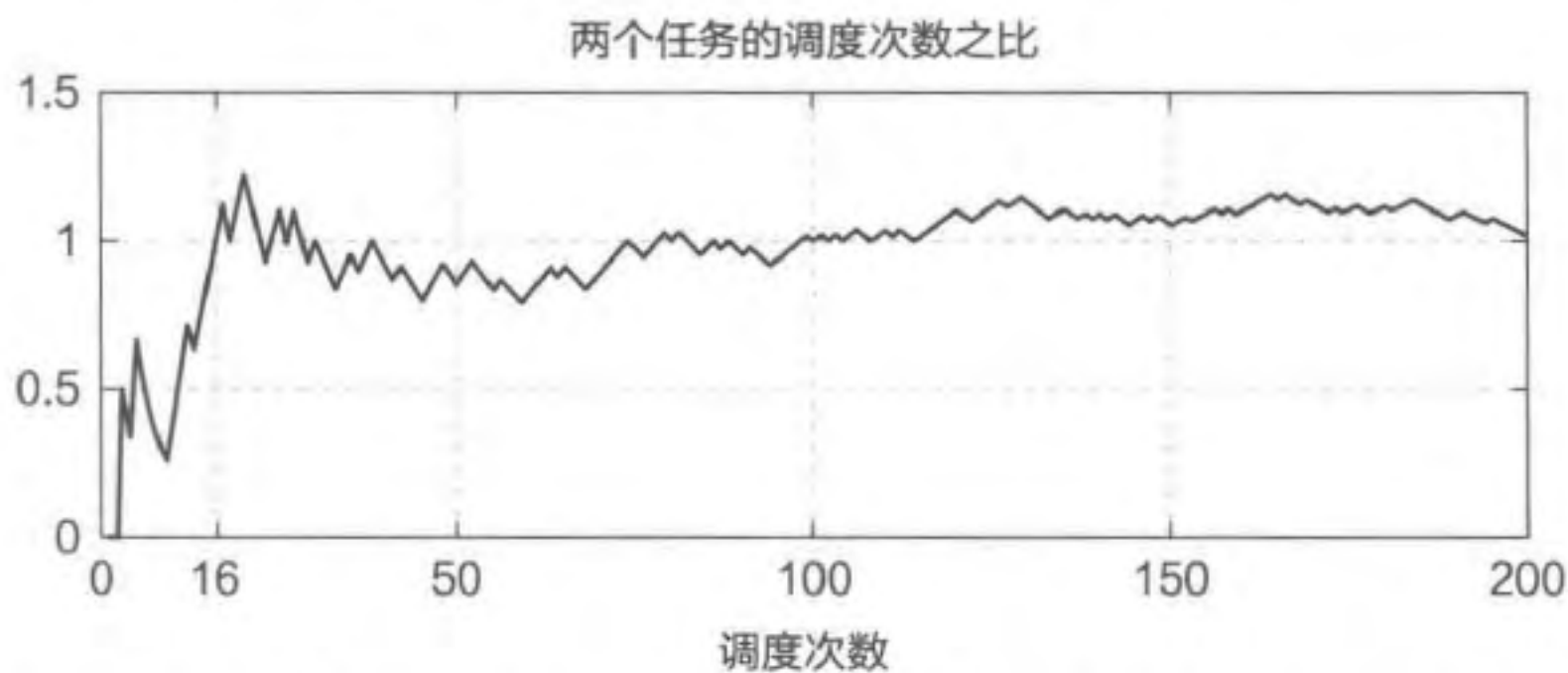


图 7.14 两个相同份额的任务在彩票调度下被调度次数的比例变化

7.5.2 步幅调度

本节将介绍确定性的公平共享调度策略——步幅调度 (Stride Scheduling)^[4]。该策略与彩票调度非常相似，也会使用份额的概念为每个任务分配占用 CPU 时间的比例。不同点在于，步幅调度采用了确定性的方式调度任务，核心概念是步幅 (Stride)。

假设在一台机器上存在任务 A 和任务 B，它们的份额之比为 5:6，那么怎么使这两个任务使用的 CPU 时间之比也是 5:6 呢？调度器需要统计不同任务相对使用 CPU 资源的情况，并选择相对使用 CPU 资源最少的任务执行。为了统计 CPU 资源的相对使用情

况，一种直观的方法是使用任务真实执行的物理时间统计。进一步假设任务 A 执行了 60 秒，任务 B 执行了 66 秒。为了确认谁获得了超过给定份额的 CPU 资源，通过计算得知，它们的执行时间之比为 10 : 11，大于份额之比 5 : 6。因此，可以确认虽然任务 A 物理上使用了较少的 CPU 时间，但实际上获得了超过预期的 CPU 资源。但该方法的问题是无法直观体现任务相对占用 CPU 资源的情况。由于系统中往往有大量的任务，甚至任务的份额会动态变化，为了比较 CPU 资源的相对使用情况，所需的计算会变得非常复杂。为了简化上述计算，虚拟时间（Virtual Time）的概念被引入。这是指将物理时间除以任务对应的份额，从而通过比较两个任务的虚拟时间，直接获得 CPU 资源的相对使用情况。根据上述例子，任务 A 的虚拟时间是 12，大于任务 B 的虚拟时间是 11，因此任务 A 相对占用了更多的 CPU 资源。

由于任务的虚拟时间越少则越应该获得 CPU 资源，一般使用虚拟时间的调度策略会选择调度所有任务中虚拟时间最小的任务。现在，每个任务都会记录它已经执行的虚拟时间，并且仅在任务被调度时更新。同时，将它们的运行时间片设置为相同的 T 秒，任务 A 每次被调度时，其虚拟时间加 6 秒（步幅为 6），任务 B 每次被调度时，其虚拟时间加 5 秒（步幅为 5）。

图 7.15 是上述例子的示意图。每个方框代表任务执行的虚拟时间长度，方框内的数字代表任务被真实执行时的调度次序。步幅调度告诉系统在某一时刻应该调度哪个任务，并且这个调度是可以保证公平共享的，即每当这两个任务的虚拟时间增加 30 秒时，它们被调度的次数之比为 5 : 6，所使用的 CPU 时间之比也是 $5T : 6T$ ，即 5 : 6。换句话说，通过步幅计算任务的虚拟时间，可以保证任务被调度的次数与份额呈正比。至此，我们给出了步幅调度的基本算法，由于所有的计算都是确定性的，因此最终的调度也是确定性。

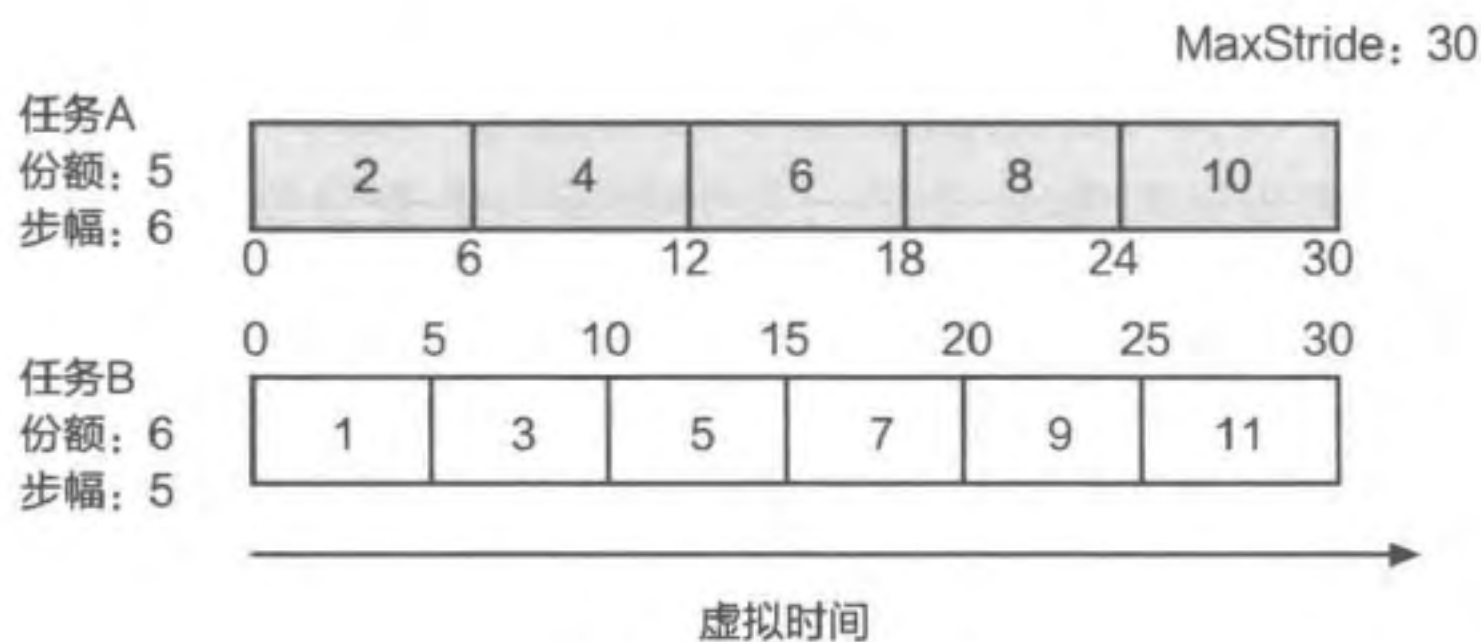


图 7.15 步幅调度示例

步幅调度通过设置虚拟时间的方式，使得任务在每次调度时增加一定的虚拟时间，即步幅。所经历的虚拟时间相同的任务，它们使用的 CPU 时间之比就是步幅的倒数之比。换句话说，任务的份额之比正对应了任务的步幅的倒数之比。

代码片段 7.3 给出了步幅调度的伪代码，对应 7.1.2 节描述的编程接口（代码

片段 7.1)。在从运行队列选择下一个需要调度的任务时，步幅调度会选取当前虚拟时间最小的任务，将该任务移出运行队列 `run_queue` (`remove_queue_min`, 第 4 行)，并最终返回该被选择的任务 (第 6 行)。

代码片段 7.3 步幅调度伪代码

```
1 struct process* sched_dequeue(void)
2 {
3     // 选择并移除运行队列中虚拟时间最小的任务
4     struct process* proc = remove_queue_min(run_queue);
5     // 调度该任务
6     return proc;
7 }
8
9 void sched_enqueue(struct process* proc)
10 {
11     // 基于该任务的步幅 stride, 计算调度后的虚拟时间 pass
12     proc->pass += proc->stride;
13     // 将该任务重新插入运行队列中
14     insert_queue(run_queue, proc);
15 }
```

当任务被执行完一个时间片，需要放回运行队列时，需要更新该任务所经历的虚拟时间。具体地，步幅调度为每个任务维护 `pass` 变量 (第 12 行)，表示该任务的虚拟时间。使用 `stride` 变量保存每个任务的步幅 (第 12 行)。在调度结束后按步幅增加其虚拟时间 (第 12 行)，并将该任务重新加入运行队列 `run_queue` 中 (`insert_queue`, 第 14 行)。为了使 `stride` 是整数，会设置一个极大的整数 `MaxStride` (图 7.15 为了直观，设置 `MaxStride` 为任务 A 和任务 B 的份额的最小公倍数)，并根据公式 $\text{stride} = \text{MaxStride} / \text{ticket}$ 计算 `stride`，其中 `ticket` 就是份额。

在真实系统中需要注意的是，由于任务可能在任意时间进入系统，因此任务的 `pass` 不能简单地从 0 开始设置，而应该设置为当前所有任务的最小 `pass` 值。否则，可能会导致新进入的任务长时间占用 CPU。

7.5.3 份额与优先级的比较

上文介绍了两种公平共享调度策略：彩票调度和步幅调度。彩票调度使用随机的思想，因此设计和实现会相对简单，但可能无法保证完全公平，特别是在任务运行时间较短、调度次数不足的情况下。相对地，步幅调度则完全保证了任务的公平共享，但代价是需要额外的维护开销。

此外，在优先级调度策略和公平共享调度策略中，优先级和份额都体现了任务的重要性，它们的差异直接导致了两种调度策略适用场景的不同。优先级与份额都表示任务在系统中的重要程度，但是它们的目的是不同的。优先级调度是为了优化任务的周转时

间、响应时间和资源利用率而设计的。不同任务的优先级只能用于相互比较，表明任务执行的先后。而基于份额的公平共享调度是为了让每个任务都能使用它应获得的系统资源。份额的值明确对应任务应使用的系统资源比例，而不同任务的份额可以用于计算这一比例。因此，在优先级调度策略和公平共享调度策略中进行选择时，必须明确当前目标场景更适合优先级还是份额。

7.6 多核处理器调度机制

本节主要知识点

- ❑ 如何维护多核处理器的运行队列？
- ❑ 如何均衡多核处理器的负载？
- ❑ 如何指定运行某个任务的 CPU 核心？

在之前的章节中，我们主要讨论了单核场景下的处理器调度策略。在该场景下，调度器只需要选择合适的任务让唯一的 CPU 核心执行即可。然而，随着多核处理器的出现，处理器调度也变得更加复杂。在多核场景下，调度器不仅需要选取合适的任务，还需要分配合适的 CPU 核心用于执行任务。为了支持多核处理器调度，调度器机制也要进行对应的扩展，本节将主要介绍多核处理器调度的机制。由于篇幅原因，更多有关多核处理器调度的内容将在本书的在线部分进行介绍。

7.6.1 运行队列

一种扩展单核处理器调度机制的直观思路是让所有 CPU 核心共享一个全局的运行队列。图 7.16 展示了基于全局运行队列的多核调度机制。当一个 CPU 核心发起调度时，根据给定的调度策略，从全局运行队列中选择下一个由它执行的任务。当任务的时间片耗尽后，任务会被重新放回全局运行队列中。

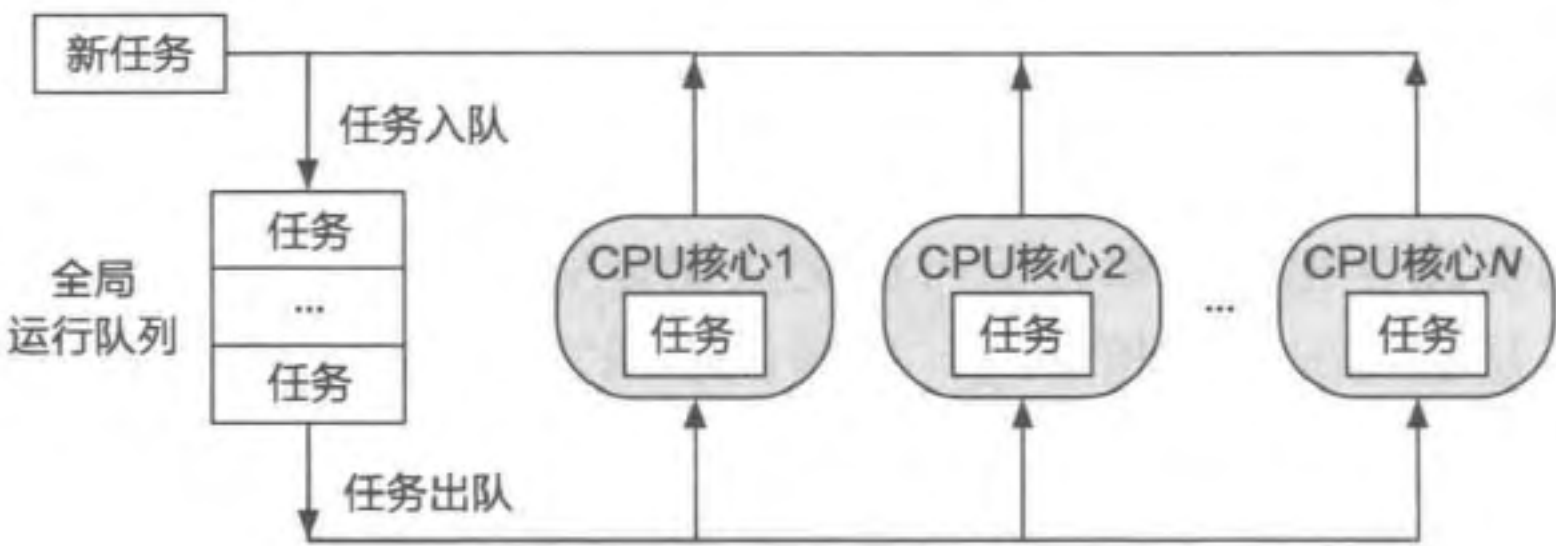


图 7.16 基于全局运行队列的多核处理器调度

全局运行队列是一种简单直观的多核调度机制，但也会导致许多性能方面的问题。